

on project using the 2FA Verification template and follow all instructions within it. A link to this template should be in the notes for the organization in Zendesk.

2. Contact Through Slack (skip if no shared Slack channel)

1. Within the customer's shared Slack channel with us, use the template below to alert them to the fact that a new 2FA disable request exists in their account management issue tracker. Be sure to replace the following variables:

- SLACKUSERNAME - The Slack handle of a user that is authorized to allow GitLab Support to disable 2FA for the customer's user accounts. If there are more than one, add them as well.
- ISSUELINK - The URL of the 2FA reset issue created on the shared project

<details>

<summary markdown="span">Notify Customer - Slack</summary>

<p>Hi SLACKUSERNAME - we've received a request from one of your users to disable 2FA on their account.

<p>Could you vouch for them by following the steps in this issue: ISSUELINK?

<p>Once you've done that, please let me know. If you don't get to this within 24 hours, we'll use our standard account verification procedures to determine if they're eligible for a 2FA reset.

>Note: If the customer has created an issue using the 2FA Verification template themselves and sent us a Zendesk ticket with a link to it, skip this step.

3. Wait For Authorization

Wait for the customer to comment on the issue and approve the request to disable 2FA.

As stressed in the Slack notification template, we will wait for the customer's answer for 24 hours. If no response is received by then, regular 2FA verification will take place via the challenges workflow.

4. Disable 2FA

Once the customer has approved the request, disable 2FA on the user's account, add an Admin Note on the user's account, and then close both the support ticket and issue.

Peer review is not required. You may make the change yourself.

SECTION: support/workflows/500_errors.md

Overview

This guide provides resources for the diagnosing of 5XX errors on GitLab.com. This is used when a user contacts support stating they're receiving either a 500 or 503 error on GitLab.com.

Reports of Slowness

If reports of slowness are received on GitLab.com, first take a look at the GitLab Grafana Monitor, especially:

- Worker CPU -> Git CPU Percent
- Worker Load -> Git Worker Load

Degraded Runner Performance

If a customer reports a shared runner running slower than it normally does, it is likely there is a degraded performance happening during the period the customer experienced slowness on the pipeline.

Check the CI Runners Overview graphs where you will find an increase in queue apex and latency.

Check on the feedalerts, production, and incidents Slack channels to ensure this isn't an outage or infrastructure issue.

If you notice slowness yourself on GitLab.com

Before you post to production or make an issue, here are some helpful ways to capture data that help to narrow down the issue(s):

1. You can add performancebar=flamegraph query parameter to generate a CPU flamegraph.
1. Use the performance bar by typing pb in your browser window. Reload the page and grab the information from the server side.
1. If using Chrome, open the Chrome developer tools (View > Developer > Developer Tools), reload the page, and look at the Network tab. This will show all of the requests and times.
1. If using Firefox, there is a similar network view under Tools > Web Developer > Network which will show requests and timing.

Screenshots from any of these tools will greatly help any engineers looking into the problems.

Connection Troubleshooting

If our customer is reporting problems connecting to GitLab.com, we should ask for the following:

```
shell
traceroute gitlab.com
curl https://gitlab.com/cdn-cgi/trace
curl https://gitlab.com/cdn-cgi/trace
curl -svo /dev/null https://gitlab.com
```

Reports of File Corruption

A 503 error on a merge request page may also happen if the repository is corrupted. To confirm, a push to a corrupted repository may show the following:

plaintext

data/repositories/@hashed/ee/98/ee98b34f343b4e48106fff666d12b61f23f.git/objects/f7/e7f4782) is corrupt

If the customer is reporting a similar error above, take the following steps to verify if their file server was affected:

1. Obtain the project URL of the affected repository.
1. Open the project admin page using the URL <https://gitlab.com/admin/projects/user-namespace>.
1. Locate the server of the repository by looking at gitaly-storage-name.
1. Search the GitLab Infrastructure issue tracker for any related issue.
1. If an issue is found related to the file server, post the ticket number in the issue so an infrastructure engineer can look into it.

Workflows

The following workflows will guide you on how to search Kibana and/or Sentry for the event in our logs that caused a particular 5XX error.

Searching Kibana

See the 500-specific section in the Kibana workflow.

Searching Sentry

See the Sentry workflow.

A video walkthrough of investigating 500 errors using Kibana and Sentry can be seen here (GitLab Unfiltered).

Get the results into an issue

Once results have been found in either Kibana or Sentry, do the following.

1. Gather as much information as possible. Make an internal note on the ticket including links to the logs found in either Kibana or Sentry.
1. Search the GitLab issue tracker for any duplicate or related issue.
1. Confirm if the issue is known or unknown and proceed accordingly: Issue is known or Issue is unknown.

In a Priority 1/Severity 1 situation, consider a dev escalation.

Responding to the user

Issue is known

If the issue is known it should have a corresponding issue in the GitLab issue tracker. If you found an entry in Sentry that has been converted into an issue, you should see the issue number in the header within Sentry:

Click the issue number to be taken directly to the issue where you can leave a comment to provide a link to the Zendesk ticket.

Then, respond to the user with information about the cause of the issue, provide a link to it, and invite them to subscribe to it for updates.

Issue is unknown

Issues found in Sentry

1. Convert the issue to a GitLab issue by using the "Create GitLab Issue" button on the issue page. Note: There is a known issue with the GitLab integration in Sentry that prevents this from working,

due to the number of projects under the gitlab-org group. As a result you will most likely need to create the GitLab issue manually.

1. Comment on the issue providing a link to the Zendesk ticket.

1. Add any additional labels if needed such as customer, priority and severity, and the appropriate DevOps stage.

1. Respond to the user with information about the cause of the issue, provide a link to it, and invite them to subscribe to it for updates.

Issues found in Kibana

1. Get a "short url" to the Kibana logs.

1. Create a new GitLab issue and be sure to include a link to the Zendesk ticket along with the Kibana logs.

1. Add the bug label and any others if needed such as customer, priority and severity, and the appropriate DevOps stage.

1. Respond to the user with information about the cause of the issue, provide a link to it, and invite them to subscribe to it for updates.

>Note: If a 5xx error is found in Kibana then there is a high chance that there is also a Sentry issue for it. In those cases, add the json.correlationid filter and search for the value in Sentry with correlationid:

SECTION: support/workflows/_index.md

{{< group-pages-by-category >}}

SECTION: support/workflows/abuse_report_review_request.md

This workflow is to request a review of an Abuse Report. All blocked accounts should have an admin note with a link to a relevant issue.

Process

1. If the account request for a review on an Abuse Report, create an AbuseReportReviewRequest issue in the Trust and Safety Operations Issue Tracker
 - The (GitLab user lookup app in Zendesk will show the admin notes for the user if they have contacted support using the email address associated with their account. Alternatively -
 - If you have access to ChatOps you can use the below command in any chatops enabled Slack channel to read admin notes for the user
> /chatops run user find <username or email>
1. If the Abuse Report Review Request is related to the abusive activity,
 - If the request does not have links to the specific commit, audit event, user action etc, request more information using the NeedMoreInformation blurb.
 - If the user has provided enough information, then complete the AbuseReportReviewRequest as per the instructions.
1. For all other cases where no abuse report exists, including Zendesk, Twitter or slack - Complete GeneralRequest issue in the Security Operations Tracker. A member of the team will review the request within 24 hours. If the request is urgent, please reach out in the abuse Slack channel.
1. Send the Support::SaaS::Gitlab.com::Blocked Accounts::Escalated-TrustAndSafety macro for the initial response to the user.

NOTE

1. All information provided by the team is considered confidential and should not be shared unless requested.
 - If you are unsure about the information you want to share, follow up in the issue for a team member to peer review the outbound blurb.
1. The information we are able to provide are sometimes subject to Legal or further review by other members of the Org, which could potentially delay the outcome of the request.
 - These request do not have a set SLA, however the team is dedicated to getting these process as soon as possible.
 - Do not hesitate to follow up on the issue or in the abuse slack channel for an update.

An Abuse Report review request can be submitted when

1. They are verified to belong to the group/project they are requesting information about.
 - In some instances we can provide limited information to non project members who reported legitimate abusive activity.
 - The content or information regarding the reported activity/content is still available on GitLab.com.

An Abuse Report review request cannot be submitted when

1. The requesting account has been blocked.
 - If the account owner is requesting access to the account to retrieve information, follow the Reinstating Blocked Accounts workflow instead.
1. Namespace or Trademark Disputes - Workflow

1. Ownership Dispute Policy - Workflow

1. Continued Abuse from an already reported account ie: The person(s) are creating multiple accounts to circumvent a block or evade detection.

- A detailed abuse report can be sent to <abuse@gitlab.com> and must be sent from an attended mailbox.

SECTION: support/workflows/account_changes.md

Overview

This workflow focuses on the process when action is required by the Support team on behalf of a GitLab.com user.

The main situations where action may need to be taken on behalf of the user:

1. Project/Group Changes

1. Account Access Requests

1. Releasing an Email Address

1. Primary email change of an enterprise user

1. Account changes for users that cannot login due to SCIM or SAML misconfiguration

User Action First

Following our Security Policy on "GitLab's Access to Your Private Repositories", actions should always be taken by the user whenever possible.

For example, users should be deleting their own projects, but if they encounter an error with every attempt and there are no workarounds, then Support can intervene with permission.

If in doubt, please ask a Support manager to review.

Project/Group Changes

In cases where Support needs to take action on the project or group, such as for troubleshooting purposes, Support should do two things:

1. Verify the user is a group Owner or project Maintainer. Otherwise, ask the user to have an owner/maintainer contact us.

1. Ask for permission to take action. See the Asking Permission section below.

1. Consider adding an Admin Note to the group admin page if Support may need to know the action was taken in the future.

You can continue working with the original requester once an owner/maintainer provides permission if that is their preference.

Account Access Requests

In cases where a user has lost access to their account, all other options (such as SSH recovery

codes, password reset) should be exhausted first.

For unconfirmed accounts, the only account action support will typically take is an email typo fix.

Before taking any action on confirmed accounts, ensure that you have verified the account owner using the Account Ownership Verification workflow.

If ownership is verified, then:

1. Confirm permission for the changes.
1. Add an Admin Note to the user's account.

Example:

1. Removing an identity tied to the user account. The identity should also be verified in these cases.

Releasing an Email Address

Similar to Account Access Requests, if a user has lost access to their account and the account shows no activity in its history, then we can consider releasing the email address for the user to create a new account with.

We can also use this workflow when a user cannot add an email address to their account because it is on a different account and is unverified. This often happens if a user has accidentally created an account using one of the single sign-on registration methods or cannot recall creating the account.

For more information on unverified/unconfirmed accounts, please see the confirmation emails workflow.

The primary (for paid users only, all users should be able to get a new confirmation email) and secondary email (for all users until 367823 is resolved) can be released following one of the processes below.

Release an unverified secondary email address

1. Confirm the email address as an unverified secondary email address.
1. If the ticket was submitted via the ZD form, please perform an email verification.
1. Once the user has replied to confirm they own the email address, remove the secondary email from the account
1. Add an Admin Note to the user's account: 2024-01-30 | removed secondary unverified email address from the account john@xyz.com| <https://gitlab.zendesk.com/agent/tickets/>
1. Reply to the user saying that their email address is now released and they can use it to create a new account.
1. Comment on this feature request

Release an email address for an inactive account

Verify account status and ownership

Check the user's activity page:

1. If an account shows any activity tied to any type of contribution (such as snippets, or comments in a project or group), use the Account Ownership Verification workflow to verify ownership.

1. If the account shows no activity:

1. Confirm that the email address the user is trying to add exists on a different account.

1. Verify that the account shows no activity and is not a member of any projects or groups.

Additionally, confirm that the following are true:

- The user is unverified
- The user has never logged in
- The user has no data (No groups or projects)

1. If the account is verified or data exists, inform the original requestor that the email is not eligible for release. They can request an account deletion if necessary.

If eligible for email release

1. If applicable, add the new email address as a CC to the ticket and ask the user to respond to the ticket from the email address they wish to add.

1. Once the user has replied to confirm they own the email address, update the email address with +release. For example, if the email address is johndoe@example.com, then update the email address on the account to johndoe+release@example.com.

- This can be done with Admin access or via Chatops

1. Add an Admin Note to the user's account.

1. Advise the customer to add the newly released email address again to their primary account.

1. Consider commenting on this feature request

Change primary email address of enterprise users

Enterprise users cannot modify their primary email address to an email with a non-verified domain. An enterprise user can only change their primary email to an email their organization owns as per its verified domains. An enterprise user or a top level group owner can reach out to Support to request primary email address change.

Note that changing the primary email address to an email that is not part of the group domain verification will disassociate the user: the user will no longer be an enterprise user.

Request from a top level group owner

Until the issue 412966 is implemented, top level group owners are not able to change their enterprise users primary email address. They can request support to change the primary email of one or more enterprise user(s).

1. Check the Account Verification Matrix for eligibility.

1. Ask manager approval to proceed

1. Use the Account Ownership Verification workflow to verify ownership.

1. If successful:

> Greetings,

>

> Thank you, we were able to verify your identity as account owner.

>

> Could you please confirm that you would like us to change the enterprise user primary address from example@primary-email.address to example@new-primary-email address ? Replying in this ticket stating you provide permission will be sufficient.

>

> Important notice: Changing an enterprise user's primary email to an email with a non-verified domain automatically disassociates them from their enterprise group. As a result of the change, your organization will not be able to manage the user account and GitLab Support will not intervene for any reason.

1. Update the primary email address of the enterprise user(s).

1. Add an Admin Note to the user's account(s).

Request from an enterprise user that may or may not be part of the group

Enterprise users have user accounts that are administered by an organization that has purchased a GitLab subscription. This means Support is not taking action without the explicit permission of one of the top level group owners.

1. Send the snippet below as an initial answer:

> Greetings,

>

> Your account is an enterprise user account, enterprise users cannot modify their primary email address to an email with a non-verified domain. An enterprise user can only change their primary email to an email their organization owns as per its verified domains.

> Updating your primary email address to an email with a non-verified domain will automatically disassociate you from your enterprise group.

>

> If you still wish to update your primary email address, please note it will require involvement of a top level group owner. Please let us know if you wish to proceed.

1. If they answer that they wish to proceed, use the Account Ownership Verification workflow to verify ownership.

1. Ask manager approval to proceed if the primary email is the only verified email (skip this step if it's an email swap request).

1. If successful, contact Owner:

- Create a new Zendesk ticket with the top level group owner's email address as the requester (found in admin) by following this specific workflow to create ticket and user

- Apply the macro General::Outbound Contact Request that ensure the new ticket routes properly and the end-user we wish to contact receives the correct notification.

- Copy the snippet below and mark the ticket as On-hold:

> Hi,

>

> We're contacting you because we've received a request from one of your enterprise users <username and email address> to modify their primary email address to an email address with a non-verified domain. This will disassociate the user from your organization: the user will no longer be an enterprise user.

>

> As you will lose any current and future administration of the user account following this change, we are asking for your permission. Replying in this ticket stating you provide permission will be sufficient.

- Make an internal comment providing a link to the requester's ticket.

- If the group contains multiples owners, choose one owner (preferably an existing support contact) as the requester and CC the others. Limit to 5 owners if more (you can pick the owners that have the most recent Last activity in the page

<https://gitlab.com/groups/<groupname>/-/groupmembers> or/and the owner(s) that is(are) listed as Source).

1. Requester's Ticket:

- Add as an internal comment the ticket created above.

- Reply to the requester with the snippet below and mark the ticket as On-hold.

> Hi,

>

> Thanks for verifying your account with us. We are now waiting for permission from your organization to release the account by updating your primary email address.

> We will keep you updated.

1. If one of the owners approve, update the primary email address of the enterprise user by swapping with a secondary if applicable.

1. Add an Admin Note to the user's account.

Account Ownership Changes

There are some conditions under which a change of ownership may be requested by a company with a business relationship with GitLab. Our support page outlines that these processes are not available for unpaid groups.

The outcome of a successful request is a new or existing user in the namespace will have the Owner role.

Account Ownership Change Request for Paid Groups

Account Ownership Change Requests are initiated when the sole Owner of a group leaves a company and an authorized representative of the company is seeking to regain control. This process should be a last resort, and self-service options must first be explored.

If a request is received, verify:

1. Current paid subscription is applied to the namespace.

1. Sole Owner's primary email address matches company domain.

1. Requestor has a GitLab.com account. Typically this user will already be a member but is not an Owner.

Ensure that the requestor has exhausted all self-service options:

- If the existing Owner's account does not have 2FA enabled, suggest that the requestor issue a password reset to the existing Owner's account, and claim the account.
- If the existing Owner's account has 2FA enabled, suggest the requestor contact the existing Owner to request that the existing Owner provide the one time password, backup codes, or private ssh key to allow the requestor to regain access, and claim the account.

If no self-service options are viable, follow the steps below:

1. Use the Support::SaaS::Gitlab.com::Account Ownership Change Request (Self-Service Not Possible) macro.md?reftype=heads), adding the account owner or account manager in CC if possible.

1. Once you have received the requested document, verify that all of the necessary information is included. If not, follow up with the requestor to obtain any outstanding information. Once the required information has been obtained, carefully follow the next steps.

1. Assess the request to verify if the following criteria have been met:

1. Self-service options have been suggested and aren't viable.

1. The requested document is completely and correctly populated, and is on appropriate letterhead.

1. The existing Owner has not been active during the last 90 days.

1. The requestor is using an email address on the same customer email domain as the existing Owner.

1. The requestor already holds a Maintainer role within the group.

1. Independent online search analysis supports the information provided (such as: the existing Owner no longer works for the customer; the roles and positions held at the organization by the requestor and signer; and there is no indication of an internal dispute between the requestor and the existing Owner).

1. If all criteria are met: elevate the requestor to Owner role.

1. If any criteria is NOT met:

1. Create a new Group Owner Change issue in the Legal and Compliance project;

1. Add a link to the issue to the Zendesk ticket;

1. Reply to the requestor using the Legal::General macro and set the ticket to "On-Hold". If you don't receive a reply after the On-Hold ticket reverts to open (4 days), ping in the legal Slack channel.

1. After receiving approval from Legal, elevate the requestor to Owner role.

1. Add an Admin note on the group admin page.

How to elevate the requestor to the Owner role

1. Using a GitLab Admin Account, go to the requestor's 'Namespace - Group - Members' section.

1. Search for the member by name or email address; in the Max role column, change the requestor's role to Owner.

1. If the requestor is not a member of the group, then press the Invite members button at the top right, enter the requestor's email address, and set the role to Owner. Press the Invite button to save your changes.

How is permission given for troubleshooting?

The Support team will not view any private information unless required to resolve an issue. Typically, the issue is filed by the account holder (for users) or valid members of the namespace (for projects and groups) via a support ticket for troubleshooting purposes as outlined in Security Policy on "GitLab's Access to Your Private Repositories".

A Support team member may look at information on pages not explicitly mentioned in the request, but will limit the scope of the review to the minimum access required to solve any issues.

The Support team will only take action from the requester if they:

- Are a member of the namespace.
- Have a problem that requires Support to investigate.
- Provide a link to the namespace.
 - This link can come from the initial form submission or a response in the ticket.

We expect users to provide specific links in order to focus on the related views and logs while investigating an issue. For example, a request to look into a CI/CD error should include links to the relevant job logs, pipelines, and/or CI YAML file.

Any time user data needs to be downloaded (such as cloning a repository), or where secrets must be revealed (such as CI/CD Variables), to further troubleshoot, requires explicit permission before continuing. Any user data that has been downloaded for reproduction purposes must be deleted when the issue is resolved, for example with our `zd-dl-wiper` tool.

Asking Permission

Before any actions are taken, please request explicit permission to take the required action. Be as specific as possible so that there is no confusion.

Once permission is confirmed by the user, then you may proceed.

Some sample phrases:

> Could you please provide permission for Support to ... ?

or

> Could you please confirm that you would like us to ... ?

Some examples:

> Could you please provide permission for Support to re-run one or more pipelines in project xyz to investigate the issue you've described? Replying in this ticket stating you provide permission will be sufficient.

>

> Could you please provide permission for our Support Engineers to look at the CI/CD variables in the project so that we confirm they are correct? Replying in this ticket stating you provide permission will be sufficient.

>

> Could you please confirm that you would like us to exchange your primary address example@primary-email.address and secondary address example@secondary-email.address on your account? Replying in this ticket stating you provide permission will be sufficient.

Impersonating

Impersonating a user is considered performing an action as another account, impersonating will update the Current sign-in IP and Current sign-in at for the impersonated user.

When impersonating a user, the administrator account will receive a slack message from the SIRTbot app asking to confirm if the impersonation was a legit action.

The action of impersonation is in accordance with our Confidentiality Terms of the Subscription Agreement.

Account changes for users that cannot login due to SCIM or SAML misconfiguration

When a user is reaching out to support to change their usernames or delete their account in order to be re-provision by an IdP, please always revert them to self-serve options first:

Self-serve options

- Use password reset to authenticate via username/password so the user can use self-serve deletion or modify their username themselves.
- Or link the user's existing account to their SAML identity by login directly to the group using the tokenized GitLab single sign-on URL to avoid deleting the account.

Password reset not received

If a user cannot self-serve because the option "Disable password authentication for enterprise users" is enabled on the top level group, please follow the steps below:

1. Confirm "Disable password authentication for enterprise users" is enabled on the top level group.
1. Ask Owner to temporary disable the option in the group so the user can regain access.
1. If the owner refused, please go through with Account Ownership Verification workflow (for Enterprise users account changes the ownership verification must be done by a top level group owner).
1. After verification is successful, ask permissions to make the changes to the account - For deletion request, do a simple deletion (just the user) as delete user with contributions can lead to deletion of issues and merge requests on a paid namespace.

SECTION: support/workflows/account_deletion_access_request_workflows.md

Purpose

This document contains instructions on how to process each type of Account Deletion or Data

Access request. It is split into two stages; Submission Handling and Request Processing, to be followed in that order. Each request will go through the same flow from opening to completion that is outlined in the chart below. All requests must be fulfilled within 30 calendar days.

mermaid

graph TD

```
A(Valid Request Received)-->B
B(Perform Account Verification)-->E
E(User Passed Account Verification Process?)-->|Yes|I
E-->|No|H
H(Apply Account Verification Failed Label)
I(Create Delete Meta Issue)
```

Procedure

Stage 1: Submission Handling

We are only able to process Account Deletion and Data Access requests if they are submitted through the official Personal Data Request form. If a request is received via any other method, we will close it and direct the user to open a request through the form.

When a user submits a request through the form, an issue is automatically created in the Personal Account Requests Service Desk, even for invalid requests. Comments made in the issue will be emailed to the user. You will communicate with the user through the issue on the progress of their request.

The purpose of this stage is to instruct you on how to close out invalid requests.

Zendesk Submissions

When a request is received through Zendesk as a support ticket, do one the following:

- If the ticket is regarding an Account Deletion request, apply the Support::SaaS::Gitlab.com::Account Deletion Instructions - GitLab.com macro, and mark the ticket as solved.
- If the ticket is regarding a Data Access request, apply the General::Personal Data Access Request Instructions macro, and mark the ticket as solved.
- If the ticket is regarding a combination of both, use a variation of both macros above, and mark the ticket as solved.

Direct Email Submissions

If a request to the Personal Account Requests Service Desk is submitted directly via email, add the account-deletion::invalid label, add a comment to the issue with the snippet below, then close the issue.

<details>

<summary markdown="span">Request Closed - Request Submitted Directly</summary>

<p>Greetings,</p>

<p>It looks like you've emailed this request in to us directly. In order for us to best assist you please re-submit this request via our Personal Data Request form. Doing so will allow us to process your request more quickly and efficiently. This request will now be closed. We eagerly await your resubmission. Thank you!</p>

<p>Regards,</p>
</details>

NOTE: Requests sent in through the official form will include a copy of the form entries in the initial description of the issue.

Spam Submissions

If the request submitted is spam, apply the label Invalid Request::Spam, then close the issue.

Duplicate Form Submissions

If a user has submitted multiple requests via the form, apply the /duplicate quick action to the duplicate issues, and respond with the following before closing the duplicates:

<details>
<summary markdown="span">Request Closed - Duplicate </summary>

<p>Greetings,</p>

<p>It looks like you have submitted multiple requests for the same purpose. </p>

<p>I'm closing this issue in favor of 123, and we will continue processing your request on the other issue.</p>

<p>Regards,</p>
</details>

Stage 2: Request Processing

Find the appropriate workflow below to process requests submitted to our Personal Account Requests Service Desk, based on their request type.

- Deletion Requests
- Data Access Requests
- Data Export (Right to Portability)
- General Questions

As a reminder, before processing a request you should make sure that you have already Streamlined Your Workflow to make requests easier to process.

Deletion Requests

The following are the types of requests that a user can file. Click the link for each to jump to the associated workflow for processing that request.

- GitLab.com Account Deletion (delete the GitLab.com account only)
- GitLab.com Account Deletion - Deceased User (delete users data that belongs to deceased owner)
- Portal Account Deletion (delete data in customers.gitlab.com only)
- Full Deletion (delete everything across all systems)
- Marketing or Sales Account Deletion (delete marketing and sales-related data only)

Currently, only GitLab.com Account Deletion or Full Deletion requests are auto-checked upon form submission.

GitLab.com Account Deletion and Full Deletion

This workflow applies to both GitLab.com Account Deletion and Full Deletion requests. When a request is submitted for either of these types, the following form entries are verified using built-in automated checks:

- Username (must exist)
- Email address (must exist)
- Username and email must match for the same account
- Account is not part of a paid namespace
- User checked the box confirming their intent to delete (deletion requests only)

After submission, the automated checks will either fail or succeed.

plain

You Should Know:

If any of the automated checks fail, the user will receive an auto-generated response detailing the reason we are unable to process their request. The issue created will be marked as invalid with "Invalid Request Received" noted in the title. These issues are scheduled to automatically close, and the label "account-deletion::invalid" will be applied. No action is required.

Step 1: Await Challenges

If all of the automated checks succeed, the user will receive a set of Verification Challenge questions in an auto-generated response. Users have a total of 7 calendar days to respond to the challenge questions.

At this stage, do the following:

1. If the account in question is a FREE account, add the account-deletion::personal label to the issue. If the account is tied to a paid namespace with a signed contract in Salesforce (a corporate request), add the account-deletion::corporate label to the issue. If the user has added comments in their form submission that reference lost 2FA or loss of access to the email

address connected to the account, add the RequestReason::lost2fa or RequestReason::lostaccess labels, as appropriate.

1. Add the Awaiting::Challenge Answers label and wait for the user to reply with the answers to the questions. If they do not reply within 7 calendar days, proceed to No Response. If they do reply within 7 calendar days, proceed to Step 2: Evaluate.

No Response

If the user fails to respond within 7 calendar days:

1. If you have not already done so, check the form comments for an alternative email, and if it was provided, tag Privacy in an internal comment in the Issue. The user likely never received notification of the Challenge Verifications and Privacy will review to determine if they should communicate directly with the user. Please do not close the Issue until you confirm with Privacy.

1. If no alternative email was provided in the form comments, close it using the following snippet (note the 3 labels that are set: Account Verification Failed deletion request::denied and denied-reason::noresponse and 1 label is removed: Awaiting::Challenge Answers):

<details>

<summary markdown="span">Request Closed - No Response</summary>

<p>Greetings,</p>

<p>We have not heard back from you with responses to our verification challenge questions, which are required in order to verify your identity before we process your request. We will now close this request. If you still wish to proceed please feel free to submit a new request via our Personal Data Request Form.</p>

<p>Regards,</p>

/label ~"Account Verification Failed" ~"deletion request::denied" ~"denied-reason::noresponse"
</details>

Step 1.5: Blocked or Banned Accounts

If the user account is not blocked or banned, skip this section.

If the user is blocked due to a user deleting their own account (see the admin note on the account):

<details>

<summary markdown="span">Account Blocked - User initiated account deletion</summary>

<p>Greetings,</p>

<p>As an account deletion has been initiated, the account stays in a blocked state for 7 days until it is permanently deleted. New accounts with the same email address or username cannot be created during that time.</p>

<p>You will need to wait 7 days, starting the day of the deletion request, to create a new account with the same email address or username.</p>

<p>Regards,</p>
</details>

For all other blocked or banned reasons, proceed to step 2 first.

Step 2: Evaluate

Evaluate the answers to the challenge questions that the user has provided using the Account Verification workflow with a data classification of RED along with the Risk Factor (GitLab internal) for data and privacy requests to confirm if the verification passes or fails.

- If the verification passes, proceed to Step 3: Create Meta Issue.
- If the verification passes, but the account is blocked/banned, proceed to Step 2.5: Verification Passed - Unblock User.
- If the verification fails and the user is not the sole owner of the groups and projects in their account, proceed to Verification Failed.
- If the verification fails and the user is the sole owner of the groups and projects on their account, proceed to Verification Failed (Sole Owner Exception).

Account already deleted

It is possible that the user may have deleted their GitLab.com account after raising the request. In such cases it will not be possible to perform further account verification, proceed with the appropriate step below according to the request type:

- For FULL account deletion requests you should proceed to Step 3: Create Meta Issue and mark the GitLab.com removal actions as complete.
- For GitLab.com account only deletion requests respond with the snippet below and close the issue:

<details>
 <summary markdown="span">Request Closed - GitLab.com account already deleted</summary>
 <p>Greetings,</p>
 <p>We are unable to complete your GitLab.com account deletion request as the account has been deleted since your request was submitted.
 This request will now be closed.</p>
 <p>Regards,</p>

/label ~"deletion request::denied"

/close
</details>

Verification Failed

If the user fails the challenge questions, apply the Account Verification Failed and deletion request::denied labels, and respond with the following snippet before closing the issue:

<details>

<summary markdown="span">Request Closed - Verification Failed</summary>

<p>Greetings,</p>

<p>Unfortunately, the answers to our verification challenges have failed. As a result, we are unable to process your account deletion request. This issue will be closed.</p>

<p>Regards,</p>

/label ~"Account Verification Failed" ~"deletion request::denied"

</details>

Verification Failed - Sole Owner Exception

If the user fails the challenge questions, but they are the sole owner of groups and projects, we can consider approval for an exception by working through the following:

1. Validate that the user is the only owner of any top-level groups and that any projects associated with their account are in their personal namespace.
 1. Check that the user has not logged in for at least 1 year.
 1. Check that the projects and groups owned by the user have not been updated in at least 1 year.
 1. Reach out to the privacy-teamhelp Slack channel to engage a Support Manager and Privacy for review and approval (approver will add Deletion Request::Exception Approved to the issue if approved).
 1. Proceed with the deletion process.

Verification Failed - Inactive Account

If the user fails the challenge questions AND does not meet the criteria for Sole Owner Exception, we can consider approval for an exception due to inactivity by working through the following:

1. Validate that the user is on a free plan.
 1. Check that the user has not logged in for at least 90 days.
 1. Check that the repository/repositories are essentially empty (there is nothing in the user account)

The purpose is to ensure that a user account does not contain any data that has been made publicly available for use by others and that deletion of the account and the groups/projects under that account will not have a negative effect.

1. Apply one of the following labels to the Issue:

Inactive::All Conditions Met (all three conditions are met)

Inactive::Has Data (the last condition fails) requires approvals per step below

1. Reach out to the privacy-teamhelp Slack channel to engage a Support Manager and Privacy for review and approval (approver will add Deletion Request::Exception Approved to the issue if approved).

Note: If a situation requires further review, and falls outside of the above criteria for a sole owner, please reach out to the privacy-teamhelp Slack channel to discuss approving an exception with a Support Manager and the Privacy team.

If the above conditions are not met, respond with the following before closing the issue:

<details>

<summary markdown="span">Request Closed - Verification Failed</summary>

<p>Greetings,</p>

<p>Unfortunately, the answers to our verification challenges have failed, and your account appears to have had recent activity within the last year. As a result, we are unable to process your account deletion request at this time. This issue will be closed, however you may try your request again at a later date.</p>

<p>Regards,</p>

/label ~"Account Verification Failed" ~"deletion request::denied"

</details>

Step 2.5: Verification Passed - Unblock user

1. Ensure that the user has passed the challenge questions and has been properly evaluated.
1. Proceed to follow the reinstating a blocked account workflow. Generally, this means opening a Trust and Safety issue for evaluation.
1. Reply to the user to let them know.

<details>

<summary markdown="span">Account Blocked - For Security review</summary>

<p>Greetings,</p>

<p>The account which you have submitted a request for is blocked. We have submitted a request for our Security team to review. Once they have reviewed, we will provide you with an update.</p>

<p>Regards,</p>

</details>

If the account is unblocked or unbanned, let the user know, then follow the rest of the process as normal. If needed, include other information about the verification process in the next reply.

```
<details>
  <summary markdown="span">Account Unblocked by Security</summary>

  <p>Greetings,</p>

  <p>Our security team has unblocked the account. We are proceeding with the request.</p>

  <p>Regards,</p>
</details>
```

If the account stays blocked or banned, apply the Account Verification Failed and deletion request::denied labels to the issue, and close it with the following:

```
<details>
  <summary markdown="span">Request Closed - Account Blocked by Security</summary>

  <p>Greetings,</p>

  <p>At this time, we cannot delete your account because your data must be retained in order for us to comply with our legal obligations, such as protecting against illegal or fraudulent activities; infringement of IP rights; distribution of harmful or offensive content; violations of the security or integrity of a computer or network.</p>

  <p>Regards,</p>
</details>
```

Step 3: Create Meta Issue

Proceed with processing the request by doing the following:

1. Let the customer know they have been verified and that you are initiating the deletion process by leaving a comment on the issue with the following snippet. Take care to replace TYPEOFREQUEST with the appropriate type of request.

For FULL account deletion requests, if the GitLab.com account has already been deleted, the response below should be used.

```
<details>
  <summary markdown="span">Verification Succeeded - Deletion Process Started (account deleted)</summary>

  <p>Greetings,</p>

  <p>Thank you for stepping through the process with us and verifying your account ownership. We have now started the process to fulfill your full account deletion request. Please note that it takes up to 30 days for the request to be processed as it's handled by different teams.</p>
```

```
  <p>As you have already deleted your GitLab.com account please do not create a new account using the email address associated with this request as this could result in the incomplete
```

removal of your data.</p>

<p>We will let you know once the deletion process is complete.</p>

<p>Regards,</p>
</details>

Otherwise use the response below:

<details>
<summary markdown="span">Verification Succeeded - Deletion Process Started</summary>

<p>Greetings,</p>

<p>Thank you for stepping through the process with us and verifying your account ownership. We have now started the process to fulfill your TYPEOFREQUEST account deletion request. Please note that it takes up to 30 days for the request to be processed as it's handled by different teams.</p>

<p>We will let you know once the deletion process is complete.</p>

<p>Regards,</p>
</details>

1. Create a new confidential issue in the Personal Account Requests Service Desk using one of the following templates, depending on the request type, and populate the issue title with the email address of the original request:

- Deletion Meta Issue - Full for Full Deletion requests.
- Deletion Meta Issue - GitLab-com for GitLab.com Account Deletion requests.

1. Follow the instructions in the top of the template, then complete each step in the issue that begins with Support Engineer: in order.

GitLab.com Account Deletion - Deceased User

Use this workflow for requests to delete user data where the sole owner of that data is deceased. No automated checks will take place with this request type.

The requestor will receive an initial reply with a set of verification challenges and documents to provide in an auto-generated response. The requester will have 10 calendar days to provide the requested information when the request was initially made. Once they have responded with the details, follow the steps below.

IMPORTANT NOTE: If at any point you are unsure of next steps or have any questions or concerns about the information in which the requester has provided, please reach out to the privacy-teamhelp Slack channel for help.

1. Add the Awaiting::Challenge Answers label, and wait for the requestor to reply.

1. If no response is received after 10 calendar days, apply the Account Verification Failed deletion request::denied and denied-reason::noresponse labels to the issue, and close it using the following snippet:

<details>

<summary markdown="span">Request Closed - No Response (Account Deletion - Deceased User)</summary>

<p>Greetings,</p>

<p>We have not heard back from you with responses to our verification questions or the required documentation, which are needed in order to process your request. We will now close this request. If you still wish to proceed please feel free to submit a new request via our Personal Data Request Form.</p>

<p>Regards,</p></details>

1. Notify privacy-teamhelp of the request so that they can begin review of the evidence provided by the requester proving their authority to act on behalf of the deceased account owner.

1. If the user has access to the account and provided answers to the Verification Challenge questions, then evaluate answers for verification.

1. If the user fails verification, please reach out to the privacy-teamhelp Slack channel for further direction.

1. If the requester fails the challenge questions, and you get instruction from privacy-team-help to proceed to verification fail, apply the Account Verification Failed and deletion request::denied labels, and respond with the following snippet before closing the issue:

<details>

<summary markdown="span">Request Closed - Failed Verification</summary>

<p>Greetings,</p>

<p>Unfortunately, the answers to our verification challenges have failed. We are required to obtain satisfactory documentation evidencing your authority to act on behalf of a deceased account owner and to obtain some minimum information to locate the account in our system. As a result, we are unable to process your account deletion request. This issue will be closed.</p>

<p>Regards,</p>

</details>

1. If the requester passes the challenge questions, or privacy-team-help has provided instruction to continue with this request:

1. Comment on the gitlab330669 feature request to note that we're having to manually process a relevant request.

1. Let the requester know they have been verified and that you are initiating the deletion process by leaving a comment on the issue with the following snippet.

<details>
<summary markdown="span">Request Closed - Verification Pass</summary>
<p>Greetings,</p>

<p>Thank you for stepping through the process with us and verifying your authority to act on behalf of the deceased account owner. We have now started the process to fulfill your deletion request. Please note that it takes up to 30 days for the request to be processed. I will let you know once the deletion process is complete.</p>

<p>Regards,</p>

</details>

1. Create a new confidential issue using the Deletion Meta Issue - Deceased.md template, populating the title with the email address of the original request.
1. Complete each step in the issue that begins with Support Engineer: in order.

Portal Account Deletion

Use this workflow for requests to delete user data from the Portal (customers.gitlab.com).

1. In the Personal Account Requests Service Desk, create a new confidential issue using the Deletion Meta Issue - Customers Portal template, populating the title with the email address of the original request.
1. Follow the instructions in the top of the template, then complete each step in the issue that begins with Support Engineer: in order.

Marketing or Sales Account Deletion

Use this workflow for requests to delete user data from Sales or Marketing systems.

1. No confirmation or verification is required for this request type under data privacy laws. Unless the user responds back in the issue that they did not submit the request, proceed with the workflow.
1. In the Personal Account Requests Service Desk, create a new confidential issue using the Deletion Meta Issue - Marketing template, populating the title with the email address of the original request.
1. Follow the instructions in the top of the template, then complete each step in the issue that begins with Support Engineer: in order.
1. Let the requester know the deletion process is complete by leaving a comment on the original issue with the following snippet.

<details>
<summary markdown="spam">Request Closed - Deletion Complete</summary>
<p>Greetings,</p>
<p>Your request to delete your data has been completed.</p>
<p>Regards,</p>
</details>

Data Access Requests

Users can request the following to obtain information about their data. Use this workflow for requests to access full details about what data we have on the user.

- Data Access Request (provide all data from all systems GitLab has related to the user)
- General Questions (any other inquiries related to the users data)

Use the following workflows based on the type of request submitted.

1. [] Support Engineer: Ensure the user has replied from the same email on the request issue. If the user fails to respond within 7 calendar days, close the issue using the following snippet, and apply the Account Verification Failed data-access-request::denied and denied-reason::noresponse labels to the issue:

<details>

<summary markdown="span">Request Closed - No Response to Verify Email</summary>

<p>Greetings,</p>

<p>We have not heard back from you, which is required in order to validate your email address before we process your request. We will now close this request. If you still wish to proceed please feel free to submit a new request via our Personal Data Request Form.</p>

<p>Regards,</p>

</details>

1. Review the requested data by the user. If the requested data is out of scope, apply the data-access-request::denied label to the issue:

<details>

<summary markdown="span">Request Closed - Out of Scope</summary>

<p>Greetings,</p>

<p>You've opened a Data Access Request which is used specifically to obtain a copy of any personal data GitLab holds on you. In the course of filing this request, you've asked for data beyond the scope of what can be provided.</p>

<p>Usage and other out-of-scope account data can be obtained by logging into your GitLab.com account and cannot be provided through a Data Access Request. If you have specific questions about this kind of data, please review your options for support at <https://about.gitlab.com/support>.</p>

<p>Regarding this request, we can:</p>

continue the process and provide a report of all in-scope personal data.

close this request with no further action.

<p>Please let us know how you would like to proceed. </p>

<p>Regards,</p>

</details>

1. In the Personal Account Requests Service Desk, create a new confidential issue using the personaldataaccessrequest template, populating the title with the email address of the original request.

1. Follow the instructions in the top of the template, then complete each step in the issue that begins with Support Engineer: in order.

Data Export Requests (Right to Portability)

Use this workflow for data export request submissions. Note that we can only action on this if the request is for personal namespace projects, or projects in groups where the user is the only member. This workflow cannot be completed if the request indicates the user's country is Cuba, Iran, North Korea, Syria, Russia, Belaru or the Crimea, Donetsk or Luhanks regions of Ukraine as these are embargoed countries and we are not permitted under Trade Compliance laws to engage with individuals in those locations. You can visit the privacy-teamhelp Slack channel if you have any questions.

1. The user will receive an auto-response directing them to the project export steps for self-serve. Keep this issue opened for 7 days and close after that time if there are no further updates. Apply the Export Request::self-serve label.

1. If the user is located in an embargoed country they will receive an auto-response informing them that GitLab is unable to fulfill their request and the Issue will be auto-closed

1. If the user runs into any issues using the self-serve steps, they can reply to the auto-response with more details about the error they are receiving. If they reply after the issue has been closed, re-open the issue.

1. Before we continue to assist, we must validate their identity. Manually send the customer the Verification Challenge questions through the issue. Users have a total of 7 calendar days to respond to the challenge questions.

1. Follow the same steps we use for account deletion requests to validate their account:

- Await challenges
- Evaluate answers

1. Only if the user passes account validation, start the process to get them project exports.

1. If the user cannot sign in, export project using the UI or the API.

1. If there are errors, follow the project exports workflow. You can also search ZenDesk for examples of past project export tickets from paid customers for additional troubleshooting help and ideas.

1. Upload the project exports to a new folder in the "Personal Data Requests" shared drive. Share the request specific folder with the user and let them know they should download the files within 7 days.

1. After 7 days, unshare the request related folder and ask them to confirm they have the files.

1. Once their issue is resolved, close the issue and apply the Export Request::complied label.

Important Note: Because we can not deny free users from support to export their data if they run into a problem, we will need to assist them through the issue (not ZenDesk).

General Questions

Use this workflow for general question submissions.

1. If the question is something Support can answer, respond to the user's inquiry.
1. If needed, engage a Support Manager or the Privacy team for guidance and messaging. You can visit the [privacy-teamhelp](#) Slack channel to ask for help.
1. Close the issue once the users question(s) have been answered.

Exceptions

While processing a request, certain scenarios may arise that necessitate the escalation of the request to the Privacy team. The most common of these scenarios can be found in the Privacy Escalation Meta Issue template. If a request needs to be escalated to the Privacy team, do the following:

1. Create a new related issue using the Privacy Escalation Meta Issue template.

If you're ever unsure if a particular scenario requires escalating, reach out to the Privacy team via the [privacy-teamhelp](#) Slack channel.

References

- Account Deletion & Data Access Requests - Introduction

SECTION: [support/workflows/account_verification.md](#)

Overview

This workflow covers how a user can provide account verification. It should be used for any account changes.

Account actions can only be completed if the workflow below is successful.

Related topics

User Account Verification

GitLab.com requires that users without 2FA enabled verify their email address when a login attempt meets certain high-risk criteria. For users requesting support when their email address is no longer valid and are unable to receive the email, please see the [Email account lost](#) workflow.

If the user has questions about the email, phone, or credit card verification GitLab is requiring, please refer to the [Internal Handbook](#) for the workflow to follow.

GitLab Team Members

If the user is a GitLab team member, have them contact IT Ops.

2FA removal within GitLab

See the 2FA Removal workflow for instructions to process 2FA removal requests.

Workflow

The workflow applies to all cases where account verification is required, aside from 2FA removals.

Keeping the Ticket Simple and Accurate

Because an ownership-verification ticket is a matter of record, the ticket must be simple, accurate, and tightly focused on the access issue.

Do not allow the customer to bring up unrelated topics.

Step 0: Ticket metadata

Ensure that the ticket form, category, subcategory, and topic are accurate.

Most SaaS Account categories have automations or triggers.

Account verification matrix

The table below provides a summary of the available verification options based on the owner and user type:

Requester	Target	Challenges	Support PIN	Notes
---	---	---	---	---
Enterprise Owner	Own account	Owner passes challenges on own account	Support PIN must be provided by a different owner	If no other owner is available for vouch, see internal handbook for other challenges.
Enterprise Owner	Member of paid group or intent to be added or Enterprise User	Owner or target user passes challenges on their own account	Support PIN is from requesting owner	Multiple enterprise users may be handled per ticket. Target user does not have to be CCed on ticket.
Owner	Non-enterprise user	Target user passes challenges on own account	Support PIN from owner	One user per ticket. Communication is direct from the target user who must be CCed on ticket.
Paid User	Own account	User passes challenges on own account	N/A	If owner is needed, owner must open a new ticket.
Paid User	Other member of same paid group	Target user passes challenges on own account	N/A	If owner is needed, owner must open a new ticket.
Free User	Non-member of group with intent to be added	Not allowed	request must come from group owner	Owner vouch required

See the Enterprise User section on how to identify if a user is an Enterprise User.

Step 1: Sending Challenges

If you need a basis for a response where you send the challenges, use the

Support::SaaS::Gitlab.com::Account Ownership Verification - GitLab.com macro.

Step 2: Checking challenge answers

> Note: In case the user sends back very minimal information and it's clear it's not sufficient or the answers are vague, reply asking for more information immediately after their response. You can provide some additional guidance, such as "please provide the exact date and time of the commit, not just an approximate one".

1. To verify the challenge answers, use the Zendesk GitLab User Lookup App or, for those who have admin access, check at <https://gitlab.com/admin/users/USERNAME>.

1. Use the ZenDesk GitLab Super App's 2FA Helper or 2FA validator to determine the risk factor (GitLab internal) based on the user's answers. Data classification criteria and any notes are in the GitLab Internal Handbook - Data Classification table, which is considered the source of truth.

- Challenge answers must be evaluated against a paid namespace if the user is a member of any paid namespace. If the user is not a member of a paid namespace, refer to Conditions for 2FA Reset Consideration for further guidance.

- If a group owner is answering on an enterprise user's behalf, see the Account verification matrix for which account to evaluate the answers against. Even if the Enterprise user is not a current member of the paid namespace, the data classification is RED.

- If you need to leave a comment manually (instead of through the app), use the Support::SaaS::Gitlab.com::2FA::2FA Internal Note macro to put an internal note on the ticket.

1. If verification passed: Request that your decision be peer-reviewed by another member of the team through Slack supportgitlab-com. They will perform the steps in 3a

1. If the verification failed: Move to step 3b

Step 3a: User successfully proves account ownership

This section is typically done by the peer reviewer. If needed, the peer reviewer (or approving manager) may leave an approval note, in which case the original reviewer will perform the actions.

1. If you agree with the decision, sign into your admin account and locate the username in the users table or by going to <https://gitlab.com/admin/users/usernamegoeshere>

1. Please see Account Changes workflow.

1. Under the account tab, click Edit, add an Admin Note, and save.

Step 3b: User Fails to Prove Account Ownership

> Note: Do not provide hints to answers, or let the user know which challenges they got right or wrong. That is how social engineering works!

1. If the user is unable to pass the risk factor but we have not provided all the applicable challenges, you may offer further challenges. This is true for all users, including owners.

- Most commonly, an Owner in the top level namespace (with a valid subscription) vouch is requested. Use the Support::SaaS::Gitlab.com::2FA::2FA ask owner vouch macro. See the Verifying an Owner Vouch section for more information. The originating email of this request should match a verified email of the Owner's account. If the user is an Owner, vouch must be from a

different Owner.

- Some challenges can be answered with the help of their colleagues. Use the Support::SaaS::Gitlab.com::2FA::2FA Removal Verification - GitLab.com - Failed - Ask colleagues for help macro to let them know which challenges they can try to work with their colleagues to answer.

- When we receive a subsequent response, go back to evaluating the challenges to see if they now pass.

1. If the user is unable to pass the available challenges:

1. Inform them that without verification we will not be able to take any action on the account. For 2FA, use the Support::SaaS::Gitlab.com::2FA::2FA Removal Verification - GitLab.com - Failed - Final Response macro.

1. Mark the ticket as "Solved".

Authenticating an Owner Vouch

In a paid namespace: If the user elects to have an Owner vouch for their request, apply the macro Support::SaaS::Gitlab.com::2FA::2FA ask owner vouch. This will direct the requester to have an Owner (top-level) create a Snippet with a Support-provided string. Once they have replied verifying they have done so:

1. Use your Admin or Auditor account to browse to the Snippet provided (for example <https://gitlab.com/-/snippets/2057341>)

- Verify the text of the Snippet matches the string you specified
- Verify that the author of the Snippet is an Owner and direct member of the top-level paid group

1. If the Owner passes, you may count this towards the account verification challenges.

Note: Due to this bug some group owners are not able to create snippets. In that case, we can ask the Owner to generate and share with us a Support